

Mini Project -2

Report: Human Activity Recognition using Recurrent Neural Networks

Kousar Kousar

ID-22301044

Course Code # EE-543



1-Abstract

Human Activity Recognition (HAR) through sensor data analysis is crucial for various applications. This study employs recurrent neural networks (RNNs) to classify human activities from movement signals captured by sensors. Various configurations with distinct hyperparameters were explored to achieve optimal accuracy.

2- Introduction

Human Activity Recognition involves interpreting sensor data to classify activities. This project aims to utilize RNNs to accurately classify six human activities based on sensor data. Class 1 as downstairs motion, Class 2 as jogging motion, Class 3 as sitting motion, Class 4 as standing motion, Class 5 as upstairs motion, Class 6 as walking motion. The movement signals measured by three different sensors simultaneously.

3-Terminologies

Human Activity Recognition (HAR):

HAR refers to the process of automatically identifying and classifying human activities from sensor data. It involves recognizing activities like walking, running, sitting, or standing based on patterns detected in the sensor data.

Recurrent Neural Network (RNN):

RNN is a type of neural network architecture designed for sequence data processing. Unlike feedforward neural networks, RNNs have connections that form loops, allowing information to persist. This architecture enables RNNs to capture temporal dependencies in sequential data.

Forward Pass:

The forward pass is the phase in a neural network where input data is fed into the network, and computations are performed layer by layer until the output is generated. It involves calculating the output of the neural network based on the given input data and the current set of weights.

Backward Pass (Backpropagation):

The backward pass, also known as backpropagation, is the phase in neural network training where the gradient of the loss function with respect to the weights and biases of the network is calculated. It's an essential step for adjusting the weights to minimize the loss during the training process.

BPTT:

BPTT stands for "Backpropagation Through Time. The fundamental concept of backpropagation is extended to sequences or time steps, as encountered in sequential data like time series, text, or any sequential information. RNNs maintain state information over time, and BPTT handles the gradient calculations through these time steps.

4- Activation Functions:

Tanh (Hyperbolic Tangent): It's an activation function that squashes input values to the range $[-1, 1]$. It's beneficial for RNNs as it can provide both positive and negative values, aiding in handling sequences.

Sigmoid (Logistic Function): It's another activation function that maps input values to a range between 0 and 1. It's commonly used in the output layer of binary classification problems as it squashes values to probabilities.

Softmax: It's an activation function used in multi-class classification problems in the output layer. It normalizes the output into a probability distribution over multiple class. Softmax ensures that the sum of output values across all classes equals 1, allowing the model to predict the class with the highest probability.

Cross-Entropy Loss Function:

Cross_entropy_loss: Cross-entropy is a common loss function used in classification problems. It calculates the difference between predicted and actual values, measuring the "distance" between probability distributions. For binary classification, it's calculated as $-(d * \log(y) + (1 - d) * \log(1 - y))$, where d is the true label and y is the predicted probability.

Cross_entropy_derivative: This function computes the derivative of the cross-entropy loss function with respect to the predicted values. It's essential for backpropagation in training the neural network, helping adjust the model's parameters (weights and biases) based on the error between predicted and actual values.

5- Dataset:

The dataset is loaded from the provided H5 file. It contains training and testing data (trX, trY, tstX, tstY) for Human Activity Recognition.

6- Methodology

Network Architecture

The RNN architecture consists of a hidden layer using hyperbolic tangent functions and an output layer employing sigmoid functions. Hyperparameters such as learning rates (0.05, 0.1), hidden neurons (50, 100), and batch sizes (10, 30) were explored.

- 1- **Data Loading:** The code starts by loading the data from the provided H5 file, extracting training and testing datasets (trX, trY, tstX, tstY).
- 2- **Initialization:** It initializes weights and biases for the neural network using uniform distribution within the specified range.
- 3- **Activation Functions:** Defines activation functions like tanh, sigmoid, softmax, and the cross-entropy loss function along with their derivatives.

- 4- Forward Pass: Implements the forward pass of the RNN by iterating through the time steps of the input sequences, calculating hidden states, and generating output predictions.
- 5- Backward Pass: Computes the gradients for the weights and biases during the backward pass using BPTT.
- 6- Training Loop: Trains the RNN model using specified hyperparameters (learning rates, N values, batch sizes) over a defined number of epochs. It computes training losses and validation errors per epoch.
- 7- Testing: Evaluates the trained model on the test dataset, calculating accuracy and comparing predictions with actual labels.

7- Results

The table displays the validation error and test accuracy for different configurations of hyperparameters in a neural network model.

Analyzed Top-1, Top-2, and Top-3 accuracies for correctly classified outputs and implementation of early stopping in training the neural network based on validation error.

Learning Rate	Hidden Neurons (N)	Batch Size	Validation Error	Test Accuracy	Top 1 Accuracy	Top 2 Accuracy	Top 3 Accuracy
0.05	50	10	3.837	0.166	0.166	0.333	0.5
0.05	50	30	3.837	0.166	0.166	0.333	0.5
0.05	100	10	3.899	0.1716	0.166	0.333	0.5
0.05	100	30	3.837	0.166	0.166	0.333	0.5
0.1	50	10	3.837	0.166	0.166	0.333	0.5
0.1	50	30	3.837	0.166	0.166	0.333	0.5
0.1	100	10	3.899	0.1716	0.166	0.333	0.5
0.1	100	30	3.837	0.166	0.166	0.333	0.5

7.1- Result Analysis:

Effect of Hidden Neurons and Learning Rate:

Models with 100 hidden neurons and a learning rate of 0.05 or 0.1, combined with a batch size of 10, demonstrated slightly higher performance.

7.2- Key Insights:

Certain configurations performed consistently better.

Hidden Neurons Influence: Configurations with 100 hidden neurons tended to perform marginally better than those with 50 neurons.

Overall, the choice of the number of hidden neurons appeared to have a modest impact on the model's performance, while changes in batch size or learning rate did not significantly alter the model's accuracy or validation error.

8- Top-N Accuracy Discussion

The Top-N accuracies (Top-1, Top-2, Top-3) for the test set were examined to assess the model's predictive capabilities. This analysis highlights the cases where the correct output ranked among the top predictions, offering insights into the model's confidence in its predictions.

9- Conclusion

This report highlights the stability of the recurrent neural network model in the context of HAR using movement signals, showcasing variations in specific hyperparameters and providing a foundation for further modifications in the model's architecture or training techniques.